

0.69.1. Chercher le A sur l'écran : lire — de base, ailleurs

On cherche le B, cette fois : `lire(c, l) == ALPHABET[1]`.

1. Ce qu'il faut comprendre

C'est le **0.69**, avec le **B**, parti de (4, 4) ; la recherche compare à `ALPHABET[1]`.

C'est la **version de base** : la deuxième place, seule. Le 0.69.2 met les deux ensemble.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **709 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	66 o	← la console, à l'allumage	– la mise en route de la console
main()	257 o	← démarrage	– ta fonction
VBlank	8 o	← la console, à chaque image	– compte les images, 60 fois par seconde
AttendreImage	28 o	← image()	– attend l'image suivante
AttendreVBlank	16 o	← démarrage	– attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireNombre	– attend de pouvoir écrire une case
AdresseCase	10 o	← effacer(), poser(), lire()	– trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer()	– efface des cases
EcrireNombre	33 o	← nombre()	– écrit un nombre en chiffres
CopierTuiles	40 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran
LireManette	37 o	← bouton()	– lit les huit boutons
Tuiles	176 o	← CopierTuiles	– 11 lettres sur 43

Lettres gravées : B0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

nombre()	× 4	40 o sur place, dans main()	+ EcrireNombre (33 o)
bouton()	× 5	25 o sur place, dans main()	+ LireManette (37 o)
effacer()	× 1	15 o sur place, dans main()	+ AdresseCase (10 o), EffacerCases (12 o)
poser()	× 1	13 o sur place, dans main()	+ AdresseCase (10 o)
lire()	× 1	11 o sur place, dans main()	+ AdresseCase (10 o)
image()	× 1	3 o sur place, dans main()	+ AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// voir l'onglet variables.h

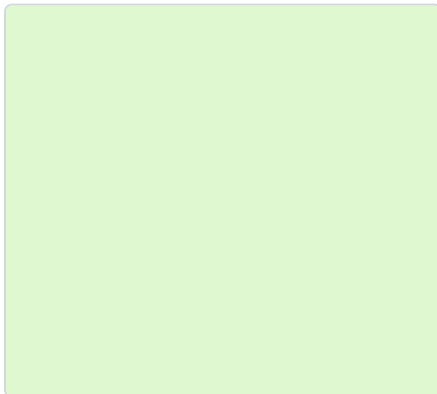
#include <effacer>      // efface des cases, ou tout le fond
#include <bouton>        // lit un bouton de la manette
#include <nombre>        // écrit un nombre en chiffres
#include <poser>         // pose une tuile sur une case du fond
#include <ALPHABET>      // les lettres de la police : ALPHABET[0] est le A
#include <lire>          // lit la tuile posée sur une case
#include "variables.h"
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

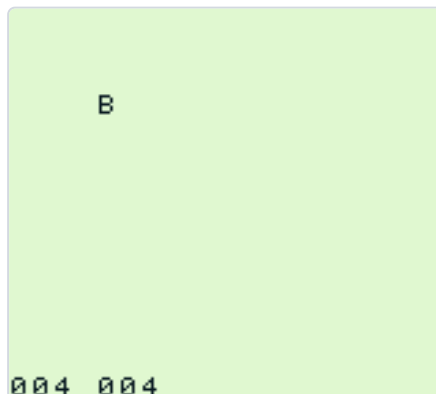
```
int main() {
    while (true) {
        image();
        if (attente > 0) attente--;
        if (attente == 0) {
            effacer(x, y, 1);
            if (bouton(DROITE)) { if (x < 19) x++; attente = 8; }
            if (bouton(GAUCHE)) { if (x > 0) x--; attente = 8; }
            if (bouton(BAS))    { if (y < 17) y++; attente = 8; }
            if (bouton(HAUT))   { if (y > 0) y--; attente = 8; }
        }
        nombre(0, 17, x);
        nombre(4, 17, y);
        poser(x, y, ALPHABET[1]);
        if (bouton(A)) {
            for (uint8_t l = 0; l < 18; l++) {
                for (uint8_t c = 0; c < 20; c++) {
                    if (lire(c, l) == ALPHABET[1]) { trouveX = c; trouveY = l; } // cherche le B
                }
            }
            nombre(10, 17, trouveX);
            nombre(14, 17, trouveY);
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



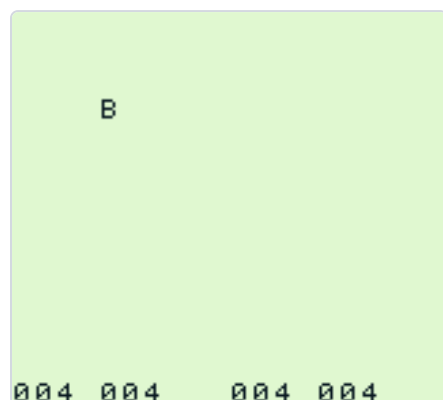
la toute première image — la console vient de démarrer — image 1



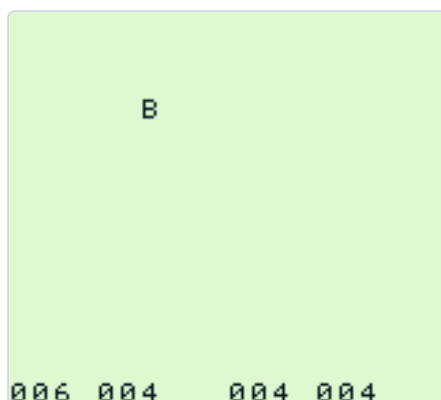
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

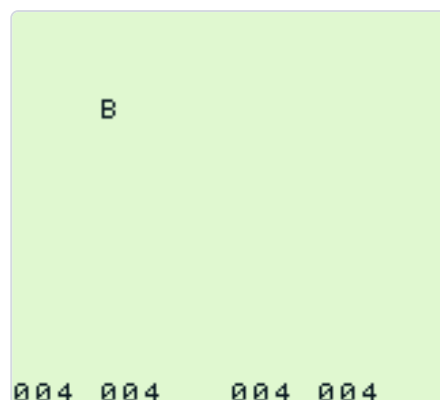
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



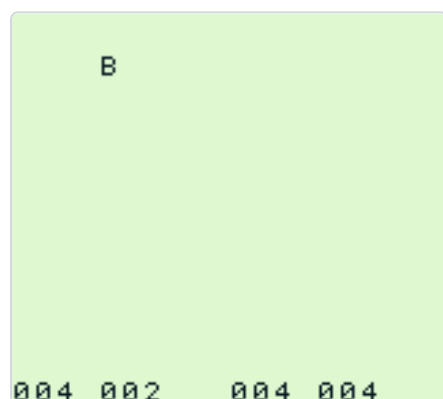
A — l'écran a changé



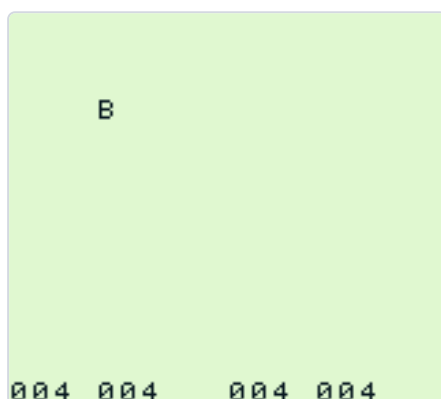
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

Un B qui bouge ; avec A, sa position trouvée s'affiche à droite.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ `lire()` a trouvé le B à sa colonne

✓ et à sa ligne

Ensuite : **0.69.2. Chercher le A sur l'écran : lire — doublé, deux positions** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++